

Retrieval-Augmented Generation (RAG) Systems

Introduction

Retrieval-Augmented Generation (RAG) combines information retrieval with large language models. Instead of relying only on the model's internal knowledge, a RAG system retrieves relevant documents from an external knowledge base and supplies them as context before generating an answer.

Architecture

A typical RAG pipeline contains document ingestion, parsing, cleaning, chunking, embedding generation, vector database storage, retrieval, prompt construction, and response generation.

Document Ingestion

Documents such as PDF, DOCX, HTML, Markdown, images with OCR, and web pages are collected. Parsing extracts raw text and metadata.

Preprocessing

Text is cleaned by removing unnecessary whitespace, fixing encoding issues, preserving headings, and optionally extracting tables and image captions.

Chunking

Large documents are split into smaller chunks. Common methods include fixed-size chunking, recursive chunking, semantic chunking, sentence-based chunking, and parent-child chunking.

Embeddings

Each chunk is converted into a dense vector using embedding models. Similar content produces nearby vectors in embedding space.

Vector Database

Embeddings are stored in vector databases such as FAISS, Chroma, Milvus, Pinecone, Weaviate, or Qdrant to enable efficient similarity search.

Retrieval

Given a user query, the query is embedded and compared with stored vectors. Top-k similar chunks are returned. Hybrid retrieval may combine dense retrieval with keyword search.

Re-ranking

A cross-encoder or reranker can reorder retrieved chunks to improve relevance before sending context to the language model.

Prompt Construction

Retrieved passages, user query, and system instructions are combined into a structured prompt while respecting the model context window.

Generation

The language model generates an answer grounded in the retrieved context, reducing hallucinations and improving factual accuracy.

Evaluation

Common metrics include Recall@K, Precision@K, MRR, nDCG, answer faithfulness, answer relevance, latency, and cost.

Challenges

Poor chunking, weak embeddings, noisy documents, context window limitations, stale data, and retrieval failures can reduce performance.

Advanced Techniques

Advanced RAG includes query rewriting, multi-query retrieval, HyDE, contextual compression, metadata filtering, parent-child retrieval, graph RAG, agentic RAG, and multimodal RAG.

Applications

RAG is widely used for enterprise search, customer support, legal research, healthcare assistants, educational platforms, code assistants, financial analysis, and internal knowledge management.

Best Practices

Use high-quality document parsing, preserve metadata, choose chunk sizes carefully, evaluate retrieval independently from generation, cache embeddings, and monitor retrieval quality over time.